DAY 26

Resilient consumers.
Happy systems.
No message left behind.

RETRY BACKOFF, DEAD LETTER QUEUE, AND POISON MESSAGE HANDLING

Keep good messages flowing. Isolate bad messages.
Recover from failures. Protect throughput.

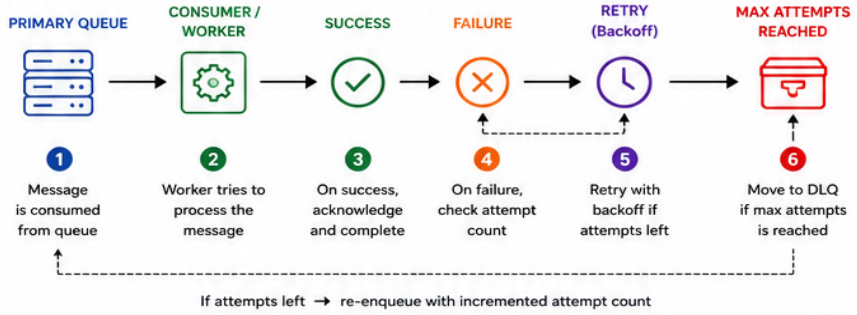
THE GOAL

Recover from transient failures, limit retries, and isolate poison messages without blocking healthy traffic.

WHY RETRIES ARE NEEDED

- Many failures are transient (network blips, dependency timeouts, overloads).
- Immediate permanent failure increases data loss risk.
- Retrying improves success rate for temporary issues.
- Without limits, retries can cause overload and cascading failures.

RETRY + DLQ FLOW (HIGH LEVEL)



BACKOFF STRATEGIES

FIXED BACKOFF

Wait a constant time between retries.

Example

Attempt 1 → wait 2s
Attempt 2 → wait 2s
Attempt 3 → wait 2s
...

- Simple and predictable.
- Can still overload failing dependency.

EXPONENTIAL BACKOFF (+ JITTER)

Wait time grows exponentially.

Example (base 2, 1s)

Attempt 1 → wait 1s
Attempt 2 → wait 2s
Attempt 3 → wait 4s
Attempt 4 → wait 8s
...

- Reduces load during sustained failures.
- Jitter prevents retry storms.

MAX ATTEMPTS & DLQ

- Some messages are poison (invalid data or persistent business conflicts).
- After bounded attempts, route message to DLQ.
- DLQ prevents one bad message from blocking throughput.
- DLQ requires triage, replay tools, and clear ownership.

POISON MESSAGE ISOLATION

- A poison message keeps failing regardless of retries.
- Moving it to DLQ isolates the failure and protects healthy traffic.
- DLQ messages can be analyzed, fixed (if possible), and replayed.
- Visibility and alerting are critical to detect and act quickly.

RETRY ENGINE BEHAVIOR

- Consume message from primary queue.
- Process message.
- Failure occurs.
- Increment attempt count.
- Wait using backoff strategy.
- Attempts left?

RETRY METRICS & OBSERVABILITY

- **Success Count** — Messages processed successfully.
- **Retry Count** — Messages retried (attempts).
- **DLQ Count** — Messages moved to DLQ.
- **Retry Latency** — Time between attempts (avg / p95 / p99).
- **Error Rate** — Failure rate per reason (transient vs permanent).

EXAMPLE DASHBOARD

SUCCESS (unique)	RETRIED
1,248	532
DLQ COUNT	**DLQ RATE**
27	2.12%

TOP FAILURE REASONS

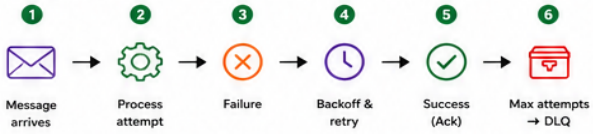
Validation Error	12 (44.4%)
Downstream Timeout	9 (33.3%)
Dependency 5xx	4 (14.8%)
Other	2 (7.4%)

DAY-26 SAMPLE: RETRY QUEUE WITH DEAD LETTER ROUTING

COMPONENTS

- **WorkMessage** — Message with id, payload, and attempt count.
- **Worker** — Business processor (can fail transiently or permanently).
- **RetryEngine** — Handles retries, backoff, and attempt tracking.
- **DeadLetterQueue** — Stores messages that exhaust max attempts.

END-TO-END FLOW



SAMPLE RUN SUMMARY

Total Messages	Succeeded (unique)	Retried	Moved to DLQ	Success Rate
1,500	1,248	532	27	83.2%

NEXT IMPROVEMENTS

- Add exponential backoff with jitter
- Add error classification (retryable vs non-retryable)
- Add DLQ replay workflow with remediation hook
- Add per-tenant / per-partition retry isolation
- Add SLO metrics for retry latency and DLQ rate
- Add alerting for DLQ spikes

> Retries are for recovery.
> DLQ is for isolation.
> Observability is for action.

**BUILD FOR RECOVERY.
ISOLATE THE BAD.
PROTECT THE GOOD.**

20 DAYS OF SYSTEM DESIGN CHALLENGE

Day 26: Retry Backoff, Dead Letter Queue, and Poison Message Handling

Day 26 focuses on how consumers recover from transient failures, cap retry attempts, and isolate poison messages without blocking healthy traffic.

It covers:

- retry policy fundamentals
- fixed/exponential backoff behavior
- max-attempt thresholds
- dead letter queue (DLQ) routing
- poison message isolation
- operational visibility and trade-offs

In short, Day 26 is about keeping event processing resilient when some messages repeatedly fail.

Core ideas

Why retries are needed

- Many failures are transient (network blips, temporary dependency overload).
- Immediate permanent failure increases data loss risk.
- Retrying improves success rate for temporary issues.

Backoff strategy

- Backoff spaces retries to avoid hammering failing dependencies.
- Fixed backoff is simple; exponential backoff is safer under sustained failures.
- Jitter is often added in production to prevent retry storms.

Max attempts and DLQ

- Some messages are poison (invalid data or persistent business conflict).
- After bounded attempts, route message to DLQ.
- DLQ prevents one bad message from blocking throughput.

Operational flow

- Main consumer processes messages and retries on failure.
- Failed messages carry attempt count metadata.
- Alerting/triage pipeline inspects DLQ messages for remediation.

Trade-offs

- More retries can improve recovery but increase latency and cost.

- Too few retries can lose recoverable work.
- DLQ requires ownership, replay tooling, and observability.

Day-26 sample: Retry Queue with Dead Letter Routing

This repository includes a small retry + DLQ simulation in both Python and Java.

Functional requirements

- Process messages from a primary queue
- Retry failed messages with bounded attempts
- Re-enqueue retryable failures with incremented attempt count
- Move exhausted messages to DLQ
- Report success/retry/DLQ metrics

High-level components

- WorkMessage: message envelope with attempt count
- Worker: business processor with simulated transient/permanent failures
- RetryEngine: retry loop and attempt accounting
- DeadLetterQueue: terminal storage for exhausted messages

Data flow

1. Engine consumes message
2. Worker attempts processing
3. On success, message is acknowledged
4. On failure with attempts remaining, message is retried
5. On failure with max attempts reached, message is moved to DLQ

Project structure

```
Day-26/
  README.md
  dlq-retry-sample/
    python/
      retry_dlq.py
      demo.py
    java/
      RetryDlqSystem.java
      Main.java
```

Run the sample

Python

```
cd dlq-retry-sample\python
python demo.py
```

Java

```
cd dlq-retry-sample\java
javac Main.java RetryDlqSystem.java
java Main
```

Interview takeaways

- Retries should target transient failures with bounded policy
- Backoff and jitter reduce cascading retry pressure
- DLQ is a reliability control, not just an error bucket
- Poison message isolation protects overall system throughput
- Observability of retries and DLQ volume is mandatory in production

Next improvements

- Add exponential backoff with jitter simulation
- Add error classification (retryable vs non-retryable)
- Add DLQ replay workflow with remediation hook
- Add per-tenant or per-partition retry isolation
- Add SLO metrics for retry latency and DLQ rate

Code Appendix

Python and Java source used in Day 26 sample.

dlq-retry-sample\python\retry_dlq.py

```
from __future__ import annotations

from collections import deque
from dataclasses import dataclass


@dataclass(frozen=True)
class WorkMessage:
    message_id: str
    payload: str
    attempt: int = 1


class Worker:
    """Simulates processing outcomes.

    - payload='ok' succeeds immediately
    - payload='flaky' fails first 2 attempts then succeeds
    - payload='poison' always fails
    """

    def process(self, message: WorkMessage) -> bool:
        if message.payload == "ok":
            return True
        if message.payload == "flaky":
            return message.attempt >= 3
        if message.payload == "poison":
            return False
        return False


class DeadLetterQueue:
    def __init__(self) -> None:
        self.messages: list[WorkMessage] = []

    def add(self, message: WorkMessage) -> None:
        self.messages.append(message)


class RetryEngine:
    def __init__(self, worker: Worker, max_attempts: int = 3) -> None:
        if max_attempts < 1:
            raise ValueError("max_attempts must be >= 1")
        self._worker = worker
        self._max_attempts = max_attempts
        self._queue: deque[WorkMessage] = deque()
        self.dlq = DeadLetterQueue()

        self.success_count = 0
        self.retry_count = 0
        self.dlq_count = 0

    def enqueue(self, message: WorkMessage) -> None:
        self._queue.append(message)

    def run(self) -> list[str]:
        history: list[str] = []

        while self._queue:
            message = self._queue.popleft()
            ok = self._worker.process(message)
            if ok:
                self.success_count += 1
                history.append(f"success {message.message_id} attempt={message.attempt}")
```

```
                continue

            if message.attempt >= self._max_attempts:
                self.dlq.add(message)
                self.dlq_count += 1
                history.append(f"dlq {message.message_id} attempt={message.attempt}")
                continue

            next_message = WorkMessage(
                message_id=message.message_id,
                payload=message.payload,
                attempt=message.attempt + 1,
            )
            self._queue.append(next_message)
            self.retry_count += 1
            history.append(f"retry {message.message_id} next_attempt={next_message.attempt}")

        return history
```

dlq-retry-sample\python\demo.py

```
from retry_dlq import RetryEngine, WorkMessage, Worker


def main() -> None:
    engine = RetryEngine(worker=Worker(), max_attempts=3)

    engine.enqueue(WorkMessage("msg-ok-1", "ok"))
    engine.enqueue(WorkMessage("msg-flaky-1", "flaky"))
    engine.enqueue(WorkMessage("msg-poison-1", "poison"))

    for line in engine.run():
        print(line)

    print("success_count:", engine.success_count)
    print("retry_count:", engine.retry_count)
    print("dlq_count:", engine.dlq_count)
    print("dlq_ids:", [m.message_id for m in engine.dlq.messages])


if __name__ == "__main__":
    main()
```

dlq-retry-sample\java\RetryDlqSystem.java

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;

public class RetryDlqSystem {
    public static final class WorkMessage {
        private final String messageId;
        private final String payload;
        private final int attempt;

        public WorkMessage(String messageId, String payload, int attempt) {
            this.messageId = messageId;
            this.payload = payload;
            this.attempt = attempt;
        }

        public String getMessageId() {
            return messageId;
        }

        public String getPayload() {
            return payload;
        }

        public int getAttempt() {
```

```
        return attempt;
    }
}

public static final class Worker {
    public boolean process(WorkMessage message) {
        if ("ok".equals(message.getPayload())) {
            return true;
        }
        if ("flaky".equals(message.getPayload())) {
            return message.getAttempt() >= 3;
        }
        if ("poison".equals(message.getPayload())) {
            return false;
        }
        return false;
    }
}

public static final class DeadLetterQueue {
    private final List<WorkMessage> messages = new ArrayList<>();

    public void add(WorkMessage message) {
        messages.add(message);
    }

    public List<WorkMessage> getMessages() {
        return messages;
    }
}

public static final class RetryEngine {
    private final Worker worker;
    private final int maxAttempts;
    private final Deque<WorkMessage> queue = new ArrayDeque<>();
    private final DeadLetterQueue dlq = new DeadLetterQueue();

    private int successCount = 0;
    private int retryCount = 0;
    private int dlqCount = 0;

    public RetryEngine(Worker worker, int maxAttempts) {
        if (maxAttempts < 1) {
            throw new IllegalArgumentException("maxAttempts must be >= 1");
        }
        this.worker = worker;
        this.maxAttempts = maxAttempts;
    }

    public void enqueue(WorkMessage message) {
        queue.addLast(message);
    }

    public List<String> run() {
        List<String> history = new ArrayList<>();

        while (!queue.isEmpty()) {
            WorkMessage message = queue.removeFirst();
            boolean ok = worker.process(message);

            if (ok) {
                successCount += 1;
                history.add("success " + message.getMessageId() + " attempt=" + message.getAttempt());
                continue;
            }

            if (message.getAttempt() >= maxAttempts) {
                dlq.add(message);
                dlqCount += 1;
                history.add("dlq " + message.getMessageId() + " attempt=" + message.getAttempt());
                continue;
```

```
                }

                WorkMessage next = new WorkMessage(
                    message.getMessageId(),
                    message.getPayload(),
                    message.getAttempt() + 1
                );
                queue.addLast(next);
                retryCount += 1;
                history.add("retry " + message.getMessageId() + " next_attempt=" + next.getAttempt());
            }

            return history;
        }

        public DeadLetterQueue getDlq() {
            return dlq;
        }

        public int getSuccessCount() {
            return successCount;
        }

        public int getRetryCount() {
            return retryCount;
        }

        public int getDlqCount() {
            return dlqCount;
        }
    }
}
```

dlq-retry-sample\java\Main.java

```
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        RetryDlqSystem.RetryEngine engine = new RetryDlqSystem.RetryEngine(new RetryDlqSystem.Worker(), 3);

        engine.enqueue(new RetryDlqSystem.WorkMessage("msg-ok-1", "ok", 1));
        engine.enqueue(new RetryDlqSystem.WorkMessage("msg-flaky-1", "flaky", 1));
        engine.enqueue(new RetryDlqSystem.WorkMessage("msg-poison-1", "poison", 1));

        for (String line : engine.run()) {
            System.out.println(line);
        }

        System.out.println("success_count: " + engine.getSuccessCount());
        System.out.println("retry_count: " + engine.getRetryCount());
        System.out.println("dlq_count: " + engine.getDlqCount());

        List<String> dlqIds = new ArrayList<>();
        for (RetryDlqSystem.WorkMessage m : engine.getDlq().getMessages()) {
            dlqIds.add(m.getMessageId());
        }
        System.out.println("dlq_ids: " + dlqIds);
    }
}
```